

ITAI 1378

Computer Vision for Artificial Intelligence

Module 02

Image Fundamentals and Processing

How computers see the world, and what we do to the pixels before any model gets to look at them.



Author: Patricia McManus

Collaborator: Claude (Anthropic)

Welcome to Module 02

Where We Are

In Module 01 we covered what computer vision is, where it came from, where it sits inside the broader AI ecosystem, and where it shows up in the real world. We looked at the five stage pipeline (acquire, prepare, extract, decide, act) at a high level, and we talked about the ethics layer that wraps every CV system.

Module 02 zooms in on stage two of that pipeline. Preparation. Before any clever model gets a chance to do anything intelligent, somebody has to feed it numbers, in the right shape, in the right range, with the right amount of detail, and without too much junk. That somebody is you. This module teaches you what those numbers actually are, how images get stored, what operations clean them up, and which tools the industry uses to do the work.

What You Will Learn

By the end of this module you will be able to:

- Explain what a pixel is, what color and grayscale really mean inside a computer, and why an image is just a grid of numbers.
- Describe the most common color models (RGB, BGR, grayscale, HSV, LAB) and pick the right one for a given task.
- Compare the most common image formats (JPEG, PNG, BMP, TIFF, WebP, HEIF) and choose the right one for storage, web, scientific, or production use.
- Apply the three categories of image processing operations: point operations, filtering, and geometric transforms.
- Recognize what filters like blur, sharpen, and edge detection actually do to pixels.
- Use OpenCV and Pillow at a beginner level, and know when to reach for which one.

Why Should You Care

Two reasons. First: image processing skills show up on almost every job description in the field. Every camera, scanner, drone, robot, medical device, and quality inspection system on the planet does image processing before anything else happens. Companies that ship CV products list OpenCV the same way other companies list Excel.

Second: image processing is where most CV bugs are born. A model that works perfectly in the lab and fails in production almost always fails because the input pixels look different from the training pixels. Wrong color space. Wrong resolution. Wrong brightness range. Too much noise. Too much compression.

You will spend more time debugging preprocessing than you will spend training models. Knowing this material well is the difference between a CV engineer who ships and one who flounders.

Part 01 | Why Image Processing Matters

Every computer vision system starts with the same problem. A camera captures light, the sensor turns that light into numbers, and now there is a giant pile of numbers sitting in memory. Those numbers are not yet useful. They are too many, too noisy, too inconsistent, and in the wrong shape for whatever the system actually needs to do. Image processing is the work of turning that raw pile into something a model, a rule, or a human can use.

A Working Definition

Image processing is any operation that takes one image as input and produces a modified image (or a measurement) as output. It sits between the camera and the model. Some of it is about cleaning up problems, like removing noise or correcting bad lighting. Some of it is about preparing data, like resizing every photo to the same dimensions before feeding it into a network. Some of it is about extracting information that the model will then use, like finding edges or counting bright spots.

It helps to compare it to working with text. If someone hands you a thousand pages of handwritten notes and asks you to extract every mention of a phone number, the first thing you do is not start reading. You scan them, type them up, remove obvious junk, standardize the formatting, and then start searching. Image processing is the same idea. Clean, standardize, prepare, then search.

Why It Matters: Three Practical Reasons

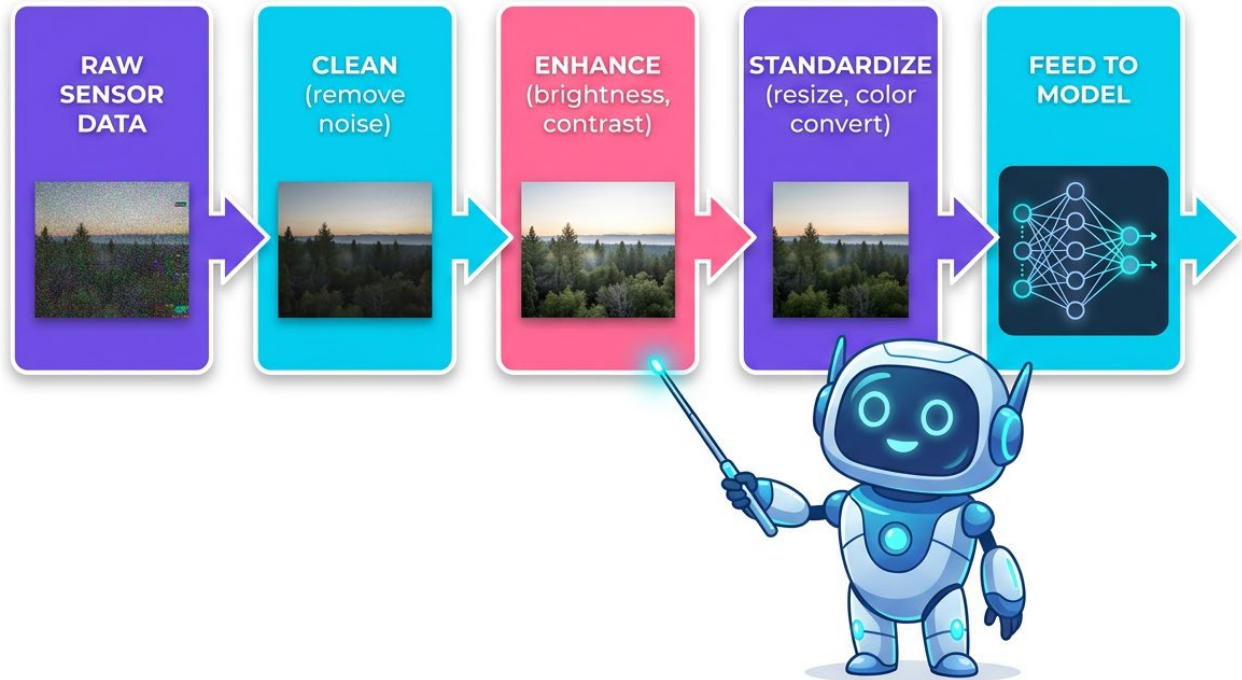
First, models need consistent input. A neural network trained on 224 by 224 pixel color photos cannot accept a 4000 by 3000 phone photo without resizing. A medical model trained on 16 bit grayscale CT scans cannot accept an 8 bit JPEG. Without preprocessing, the model is not even allowed to look at the data.

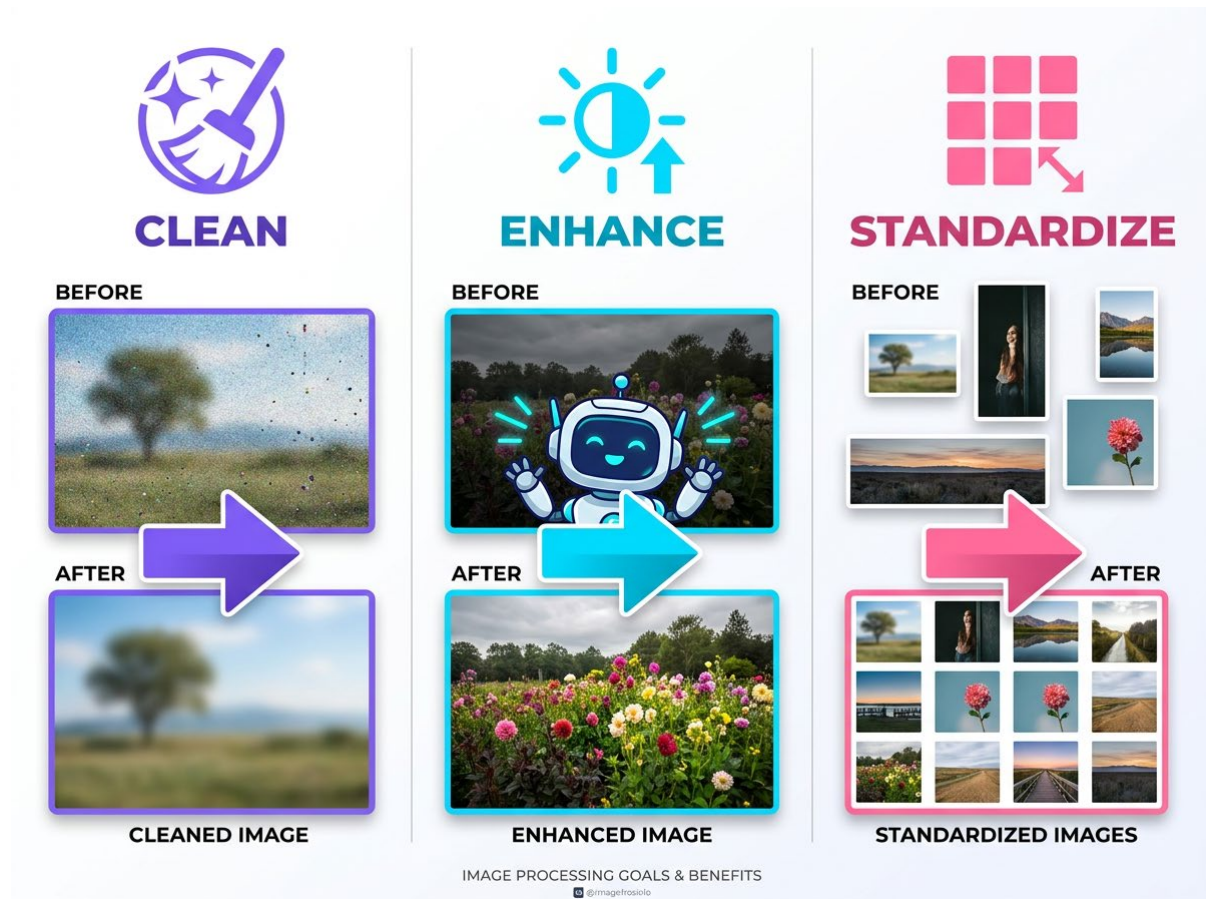
Second, garbage in produces garbage out. A model that sees blurry, low contrast, oddly colored input will produce uncertain or wrong predictions even if its architecture is perfect. The classic example is a face recognition system that works in the office but fails outside in bright sunlight, because nobody normalized the lighting before training and the test images now look nothing like the training set.

Third, smart preprocessing saves money. Modern CV models are expensive to run. If you can downsize, crop, or filter the image before it hits the model, you save compute, bandwidth, and time. In a factory

inspection line running at thousands of parts per hour, the difference between processing a 4K image and a 640 by 480 image is whether the system can keep up at all.

IMAGE PROCESSING PIPELINE





Three Goals of Image Processing

Almost every image processing step falls into one of three goals.

Goal 1: Clean

Remove things you do not want. Sensor noise. Dust on the lens. Compression artifacts. Lighting glare. Motion blur. The cleaner the input, the better any downstream operation will work.

Goal 2: Enhance

Boost what matters. Stretch the contrast so dark and bright regions both become visible. Sharpen edges so boundaries become clear. Adjust gamma so the midtones reveal more detail. Enhancement does not add information; it makes the information that is already there easier to see and to compute on.

Goal 3: Standardize

Make every image consistent. Same dimensions, same color space, same value range, same orientation. Standardization is not glamorous, but it is the single biggest reason production CV systems work or fail. A

model trained on standardized data and fed standardized data will perform reliably. The moment that consistency breaks, accuracy collapses.

KEY TAKEAWAY

Image processing is not optional. It is the bridge between a camera sensor and any useful CV system. Three goals (clean, enhance, standardize) drive almost every operation you will learn. Skip this stage and your model will fail in ways that are hard to debug.

CAREER CONNECTION | Where Image Processing Shows Up in Jobs

Every CV job description, from entry level technician to senior research engineer, includes some flavor of image processing. Quality inspection at manufacturing plants. Document scanning at insurance companies. Medical imaging pre-processing at hospitals. Real time vision on drones, robots, and self-driving vehicles. Even data scientists working on retail analytics or satellite imagery write image processing code daily. Strong OpenCV fluency on a resume is one of the fastest ways to clear an initial screen.

INDUSTRY PRACTICE | Where the Time Actually Goes

Experienced CV engineers will tell you that on a real project, model architecture decisions take days. Image preprocessing decisions take weeks or months. Companies like Tesla, Cruise, and Waymo employ entire teams whose only job is preprocessing for autonomous driving cameras. Hospitals running medical imaging AI have full time staff dedicated to standardizing scans across different machines and protocols. Preprocessing is not a small step. It is its own discipline.

KNOWLEDGE CHECK | Quick Check on Part 01

1. Name the three goals of image processing in your own words.
2. Why might a CV model that works in a lab fail in production? Give one image processing reason.
3. Why is standardization important for a model that has already been trained?

Part 02 | How Computers See: Pixels and Digital Images

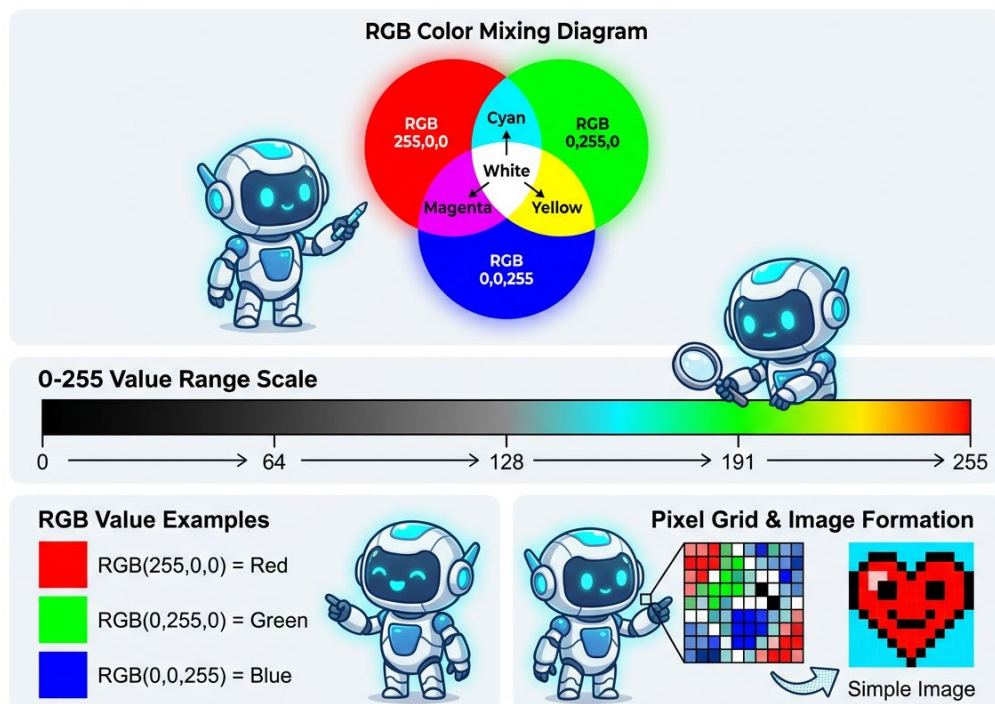
A digital image, no matter how detailed or beautiful, is a grid of numbers. Nothing more. That is the first idea this part will hammer in, because everything else in the course depends on it. Once you accept that an image is just numbers, everything that follows (filtering, edge detection, neural networks, the works) is just math being done on those numbers.

The Pixel

A pixel is the smallest unit of a digital image. The word is short for picture element. Each pixel holds a number (or a small set of numbers) that says how bright that particular spot in the image is. That is all a pixel is. A coordinate plus a brightness number.

Think of an image like a piece of graph paper. Each square is a pixel. The image is just the grid filled in with numbers. A small image might be 100 squares wide by 100 squares tall (10,000 pixels). A modern smartphone photo might be 4032 squares wide by 3024 tall (over 12 million pixels). Same idea, just bigger graph paper.

Pixel & Color Concepts



Pixel Values: The 0 to 255 Range

In the most common kind of digital image (called 8 bit), each pixel value is a whole number from 0 to 255. Zero is pure black. 255 is pure white. Everything between is a shade of gray. This range exists for a technical reason: 8 bit means each pixel uses 8 ones-and-zeros to encode its value, and 8 ones-and-zeros can hold 256 possible combinations (the numbers 0 through 255).

Why 256 shades and not 1000 or a million? Because the human eye cannot really tell the difference between 256 shades of gray and 1000 shades of gray in most viewing conditions. 256 is enough for displays, web images, photos, and almost everything you will work with in this course. Scientific and medical imaging often uses 16 bit images (0 to 65,535 shades), because subtle differences in tissue density or biological samples matter, and the extra precision is worth the bigger files.

MATH MOMENT (optional) | How Big Is an Image?

You can work out the number of values in any image with one simple calculation. Multiply width by height by number of channels.

A grayscale 640 by 480 image holds 640 times 480 equals 307,200 numbers.

A color HD image at 1920 by 1080 with three channels (red, green, blue) holds 1920 times 1080 times 3 equals about 6.2 million numbers.

A 4K color image (3840 by 2160 by 3) holds about 25 million numbers per single frame.

Now imagine 30 of those per second from a video stream. That is 750 million numbers per second. This is why CV systems care about resolution. Every extra pixel multiplies the workload.

Resolution and Dimensions

Resolution is just the width and height of the pixel grid. We write it as width by height. A 1920 by 1080 image is 1920 pixels wide and 1080 pixels tall. The product is the total pixel count, often expressed as megapixels (a 12 megapixel camera produces images with about 12 million pixels per shot).

Higher resolution means more detail captured, but also more storage, more memory, more compute. Most CV models are trained on resolutions much smaller than what your phone produces. The classic ImageNet competition used 224 by 224 pixel images. Modern detection models often work at 640 by 640 or 1280 by 1280. The resolution you feed a model is one of your most important design decisions.

⚠ MISCONCEPTION ALERT | More Pixels Is Always Better

False, and the mistake is everywhere. Higher resolution captures more detail, but only some tasks benefit. For face recognition at airport gates, 640 by 480 is plenty. For inspecting microchip wires for defects, you need many more pixels per inch. For training a generic image classifier, going from 224 by 224 to 512 by 512 might give you only one percentage point of accuracy improvement while quadrupling the compute cost. Higher resolution is not free. Pick the resolution that matches the smallest detail you actually need to see.

Grayscale Images: One Number Per Pixel

A grayscale image is the simplest case. One brightness value per pixel. Zero is black. 255 is white. Every shade of gray sits somewhere on that line. When you load a grayscale image into Python, you get a two dimensional grid (rows by columns). If the image is 100 by 200, you get a grid with 100 rows and 200 columns of numbers.

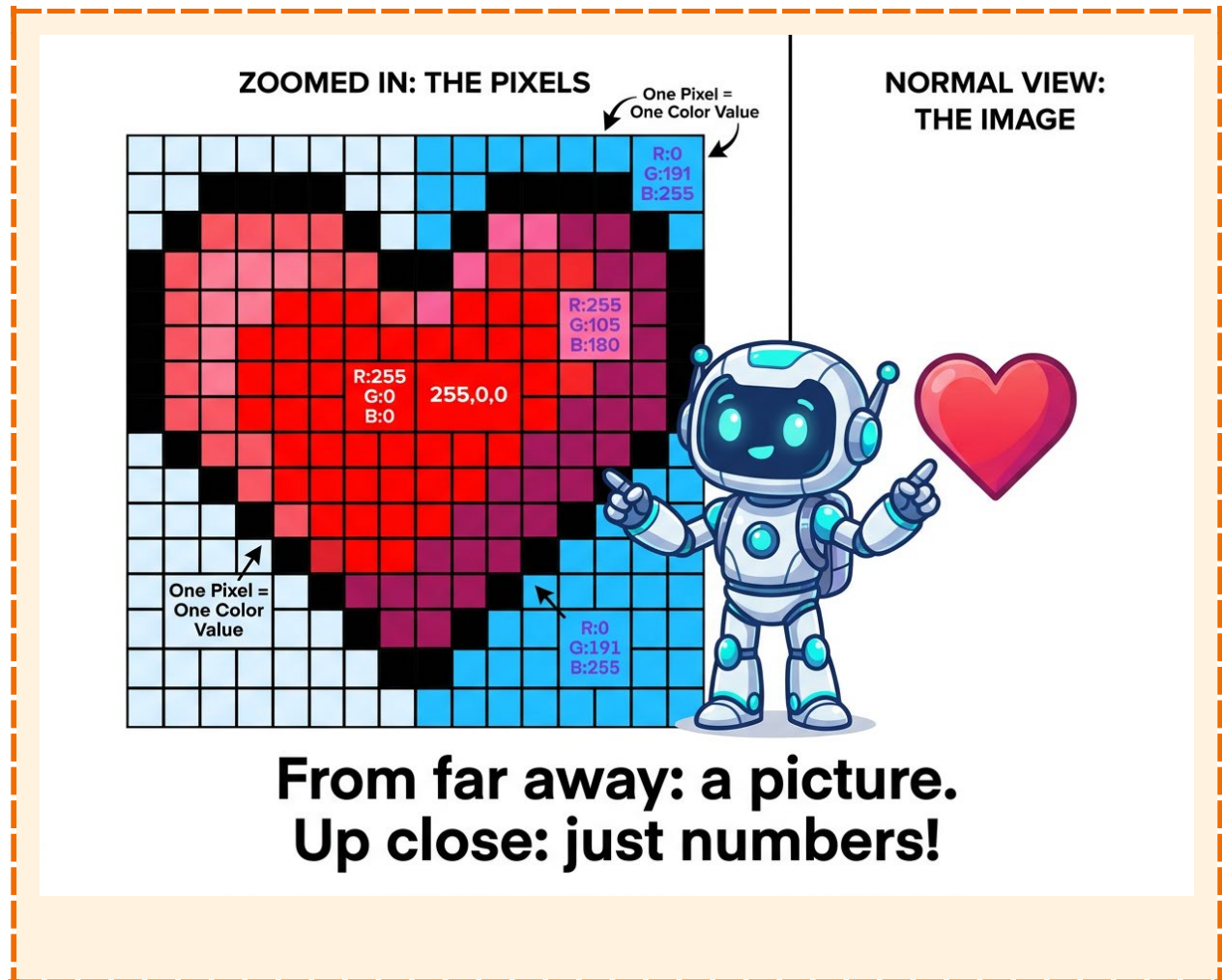
Grayscale matters for CV because many tasks do not need color. Document scanning. Most medical imaging. Edge detection. Many quality inspection systems. Working in grayscale uses one third the memory of color and is faster to process. Some classic algorithms even start by converting color images to grayscale because the algorithm only cares about brightness patterns, not hue.

Color Images: Three (or More) Numbers Per Pixel

A color image is just three grayscale images stacked. One for how much red light is at each pixel. One for how much green. One for how much blue. We call these channels. When you load a color image you get a three dimensional grid: rows, columns, channels.

For a 100 by 200 color image, you get 100 rows times 200 columns times 3 channels equals 60,000 individual numbers. Each pixel has three values describing its color. Pure white is (255, 255, 255). Pure black is (0, 0, 0). Pure red is (255, 0, 0). Pure green is (0, 255, 0). Pure blue is (0, 0, 255). Everything else is a mix.

Some specialized imaging adds more channels. Satellite imagery commonly has four (red, green, blue, near infrared) or more. Medical imaging may add depth, time, or multiple wavelengths. The pattern is the same: more channels means more numbers per pixel, and your code has to know how to read them.



Bit Depth: How Many Possible Values

Bit depth is how many possible values each pixel can take. 8 bit means 256 possible values (0 to 255). 16 bit means 65,536 possible values. 1 bit (pure black and white, no gray) is sometimes used for fax machines and barcode reading. 32 bit is used in scientific computing when precision really matters.

For most workforce CV jobs, 8 bit is standard. You will mostly work with 8 bit images. You will occasionally see 16 bit when working with medical, satellite, or scientific data. Higher bit depth means more storage, more compute, and usually more accurate results when the application needs it.

 INDUSTRY PRACTICE | What CV Engineers Actually Check First

When a CV pipeline behaves strangely, the experienced first move is to check the image properties before doing anything else. Width and height. Number of channels. Bit depth. Value range. Are values 0 to 255? Are they 0 to 1? Are they signed or unsigned? Is the channel order RGB or BGR? Most production bugs are explained in the first five seconds of looking at these basics. Most students never check. Make this your habit early.

 THINK ABOUT IT

Take a phone photo right now. If it is 12 megapixels (12 million pixels) and full color, your phone produced 36 million numbers in the time it took the shutter to click. The lens collected the light, the sensor counted photons, the image processor converted them, and stored those 36 million numbers as a file. All of that happened in about one tenth of a second. Computer vision is built on top of that miracle of engineering. Worth pausing to appreciate the scale.

 KNOWLEDGE CHECK | Quick Check on Part 02

1. What is a pixel? Define it in one sentence.
2. How many numbers are stored in a 200 by 300 grayscale image? How many in a color version of the same image?
3. What does '8 bit' mean for an image?
4. If you load a color image into Python and the array shape is (480, 640, 3), what do those three numbers represent?

Part 03 | Color Models

Color seems simple. You see red. The computer stores red. Done. Except color is not one thing. There are different ways to describe a color, and each way works better for different tasks. A color model is just a system for representing colors with numbers. This part covers the five color models you will meet in practice.

RGB: Red, Green, Blue

RGB is the model your screen uses. Every pixel on your monitor has three tiny lights, one red, one green, one blue. By varying how bright each light is, the screen produces any color you can see. White is all three at maximum. Black is all three off. Yellow is red and green at maximum, with blue off. The number trio (R, G, B) tells the screen how much of each color to mix.

RGB is the default color model for almost every digital image. Photos, screenshots, PNGs, JPEGs, web images. Almost always RGB. When you load an image in Pillow, you get RGB. When you display an image on a screen, the output is RGB. This is the model you will use most often.

BGR: OpenCV's Confusing Choice

Here is the gotcha that catches every new CV student. OpenCV, the most widely used computer vision library in the world, loads color images as BGR (blue, green, red) instead of RGB. The order is reversed. Same image, same colors, same numbers, just stored in the opposite channel order.

The historical reason is technical and old (it had to do with how Windows and certain camera makers stored color in the 1990s). The practical reason matters more: if you do not know about this quirk, your code will produce broken colors. A photo of a red apple will display as blue in OpenCV unless you convert it. Always remember: when an image enters OpenCV it is BGR. When you display it with another library like Matplotlib, you need to convert it back to RGB first.

MISCONCEPTION ALERT | OpenCV Is Wrong, RGB Is Right

OpenCV is not wrong. It is just consistent with its own internal layout, which made sense when the library was first written. The lesson is not that one is right and the other is wrong. The lesson is that channel order matters, and you must always know which order your library is using. The line `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` will be one of the most frequently typed lines in your CV code. Get used to it.

Grayscale

Grayscale is the simplest color model: one channel only. Every pixel holds a single brightness value. Many CV tasks can be done in grayscale alone, especially when color is not informative. Document scanning. Edge detection. Many forms of object tracking. Pattern matching. Working in grayscale uses one third the memory of RGB and is one third the compute. If your task does not need color, drop it.

Converting color to grayscale uses a standard weighted average that matches how the human eye perceives brightness. Green light contributes most (humans are most sensitive to green), red is next, blue contributes least. The formula does not matter to memorize; the libraries do it for you. What matters is knowing when to make the conversion: when color does not add information to the task, go grayscale.

HSV: How Humans Think About Color

HSV stands for Hue, Saturation, Value. It is a color model designed to match how humans describe colors. Hue is the color name (red, green, blue, yellow). Saturation is how pure or vivid the color is (a faded red versus a fire engine red). Value is the brightness (a dark red versus a bright red).

HSV is incredibly useful for any task that involves picking out a color. Imagine you want to detect all the red traffic cones in a road scene. In RGB, a red cone in sunlight has different RGB values than the same cone in shade, because shading changes all three channels at once. In HSV, the cone has roughly the same Hue value in both conditions, with the Value (brightness) changing. So you can filter by Hue alone and reliably find red things regardless of lighting.

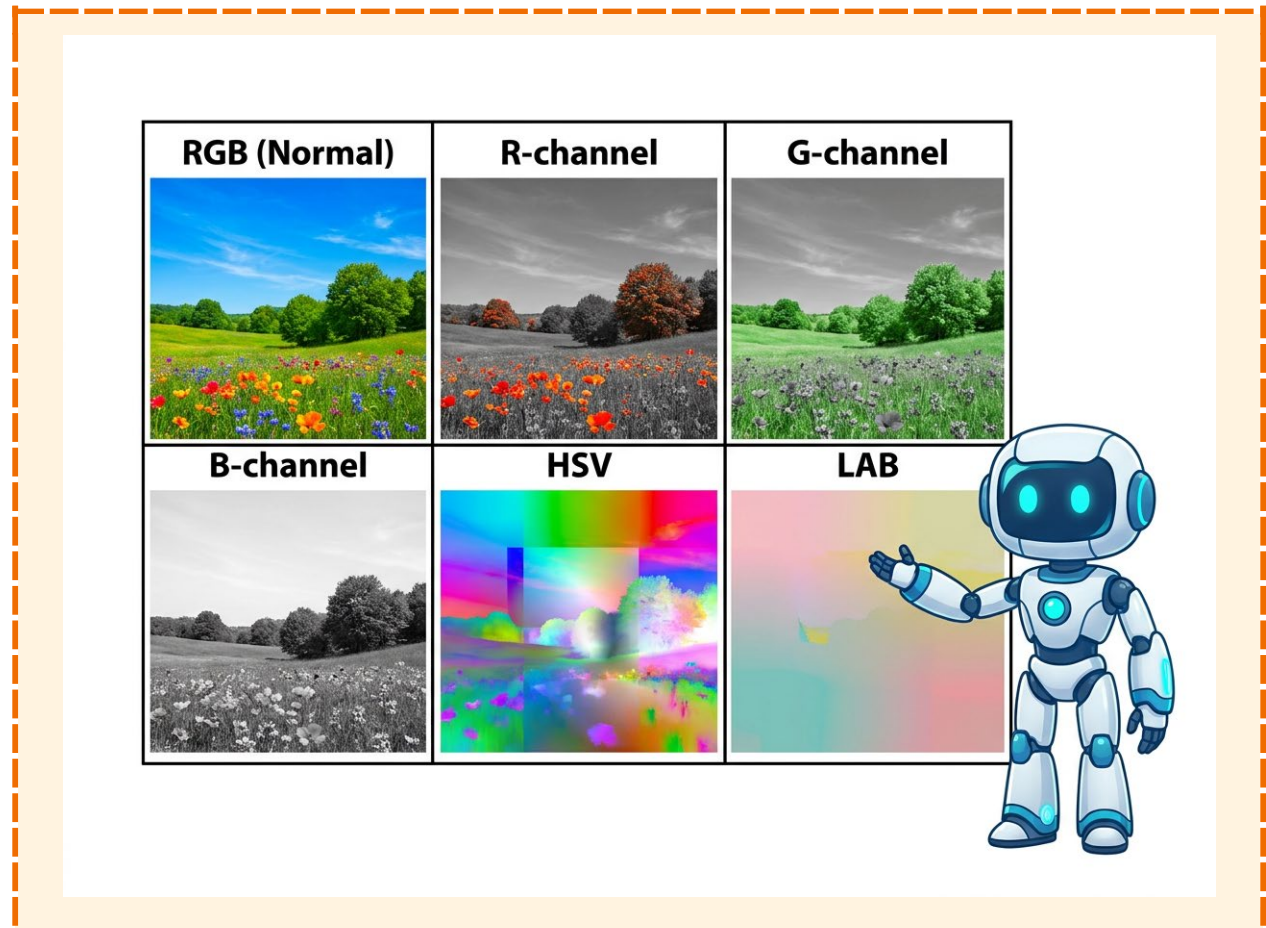
This makes HSV the go-to model for color-based filtering and thresholding. Detecting skin tones. Tracking colored balls. Picking out highlighters. Any task where you ask 'find everything in the image that is this color' works better in HSV than in RGB.

LAB: Perceptually Uniform

LAB (sometimes written CIELAB) is a more advanced color model with three channels. L is for lightness. A and B are color axes (A goes from green to red, B goes from blue to yellow). The big advantage of LAB is that equal numerical differences correspond to equal visual differences. If two colors are 10 units apart in LAB, they look about as different from each other as any other two colors that are 10 units apart. RGB does not have this property; 10 units of difference in green can look very different from 10 units of difference in blue.

LAB is the favorite of professional photo editors, color correction tools, and any system that needs to measure color similarity scientifically. In CV jobs you will see it less often than RGB and HSV, but knowing

it exists puts you ahead of most beginners. The vocabulary alone (the L channel, the A axis, perceptual uniformity) is worth carrying.



INDUSTRY PRACTICE | When To Use Each Color Model

RGB for storage, display, and most neural network inputs. Grayscale when color is not needed and speed matters. BGR if you are working in OpenCV (but convert to RGB before showing the image). HSV for color-based filtering and detection (finding all the red things, finding all the blue things). LAB for color similarity measurement and professional photo work. A working CV engineer can switch between these without thinking, and knows the right one for the task at hand.

 THINK ABOUT IT

Look around the room you are in. Pick any object. Describe its color out loud. You probably said something like 'dark red' or 'bright blue' or 'pale yellow'. You did not say 'red 217 green 32 blue 41'. You used hue, saturation, and value, even if you did not name them. HSV exists because humans already think this way. Computer scientists did not invent it. They just put numbers on what we already do naturally.

 KNOWLEDGE CHECK | Quick Check on Part 03

1. What does each letter in RGB stand for? In BGR? In HSV?
2. Why does OpenCV use BGR? Does it matter today?
3. If your job is to find all the orange traffic cones in a photo, which color model would you use, and why?
4. What does it mean to say LAB is 'perceptually uniform'?

Part 04 | Image Formats and Compression

An image format is the way an image is stored on disk or sent across a network. The pixel values are the same regardless of format (an image of a cat has the same pixels whether you save it as JPEG or PNG), but the way those pixels get packed into a file is very different. Format choice affects file size, quality, transparency support, and how the file gets used. This part covers the formats you will meet, and how to choose between them.

The Big Idea: Lossy vs Lossless

Every image format is either lossy or lossless. Lossless means the file stores the exact pixel values, with zero changes. You can save, open, and resave a lossless image a hundred times and the pixels never change. Lossy means the format throws away some information to make the file smaller. You cannot recover the lost data once it is gone.

Lossy compression is not always bad. Done well, the human eye barely notices the difference. JPEG compression is what makes the modern web possible; without it, photos would be ten times bigger and websites would load painfully slowly. Done poorly, lossy compression produces visible artifacts (blocky distortions, color smearing, soft details). For CV training data, lossy formats can quietly degrade your model accuracy if you are not careful.

JPEG: The Universal Photo Format

JPEG (also called JPG) is lossy. It is by far the most common image format in the world. Almost every photo on the internet, every camera phone shot, every email attachment of a picture is a JPEG. The format is great at compressing photographs (real world scenes with smooth gradients) and bad at compressing flat colors or sharp text.

JPEG uses a quality slider, usually 1 to 100. Higher means better quality and bigger files. 90 is typical for web photos. 95 or higher is recommended for any image you might process later. Below 80 starts to show artifacts. For CV training, never use heavily compressed JPEGs as ground truth; you are training on noise as much as on signal.

PNG: Lossless, Supports Transparency

PNG is lossless. Every pixel stored is the exact pixel you saved. Files are bigger than JPEG, but the quality is perfect. PNG also supports transparency (a transparent background), which JPEG does not. This makes PNG the format of choice for logos, icons, diagrams, screenshots, and anything with sharp edges or text.

For CV work, PNG is the safer default when you do not need maximum compression. Training data, intermediate processing results, debug visualizations, anything you might recompress later. Use PNG and you will not accidentally degrade your data through repeated JPEG re-saves.

BMP: Old, Big, and Sometimes Useful

BMP stores pixel values directly in the file with no compression at all. Files are enormous, but reading and writing them is fast and simple. BMP is rarely used today for storage or transmission, but it occasionally shows up in old industrial vision systems, scientific tools, and legacy software. If you encounter one in the wild, you can read it like any other image.

TIFF: The Professional Format

TIFF is a flexible container format. It can be lossless or lossy, can store multiple pages in one file, can support 16 bit and even 32 bit depth, can hold metadata, and is the standard in professional photography, medical imaging, scientific imaging, and document scanning. Files are larger than JPEG but the quality and flexibility are unmatched.

If you work in medical imaging, satellite imagery, or scientific computer vision, you will see TIFF often. For workforce CV jobs in business and consumer settings, you will see it occasionally.

WebP and HEIF: The Modern Successors

WebP (from Google) and HEIF (the format Apple's iPhone now uses by default) are newer formats designed to replace JPEG and PNG with better compression. A WebP file can be 25 to 35 percent smaller than the equivalent JPEG at the same quality. Both formats can be lossy or lossless, both support transparency.

These formats are becoming common in production. iPhones save photos as HEIF by default. Websites increasingly serve WebP. As a CV engineer you will need to read both formats and may need to convert them to RGB pixel arrays before feeding them to models.

MATH MOMENT (optional) | How Much Space Does an Image Take?

Take an HD color image: 1920 by 1080 pixels, 3 channels, 8 bits per channel (one byte per channel value).

Uncompressed (BMP or raw): 1920 times 1080 times 3 equals about 6.2 megabytes per image.

JPEG at quality 90: about 500 to 800 kilobytes. About one tenth the size.

PNG (lossless): about 2 to 4 megabytes depending on content. About half the size of uncompressed.

WebP at high quality: about 300 to 500 kilobytes. About one fifteenth of uncompressed.

Pick the wrong format for your storage budget and your system will run out of disk space, network bandwidth, or both.

INDUSTRY PRACTICE | Format Choice in Production CV

A production CV system might receive WebP from the front end, decode it to a raw pixel array, process it, save debug copies as PNG for engineers to inspect, and store final training data as TIFF for archival. Each format choice is deliberate. Front end gets WebP for fast page loads. Debug uses PNG so engineers can zoom in without compression artifacts. Archive uses TIFF for maximum quality preservation. A weak CV engineer treats all images as 'just images'. A strong one understands the trade-offs and picks accordingly.

CAREER CONNECTION | Format Literacy on the Job

Job interviews for CV roles often include a quick question like 'when would you use PNG versus JPEG'. Get this right and you signal that you understand more than just the trendy models. Get it wrong and you signal that you skipped the basics. The interviewer is checking whether you would accidentally lose data through bad format choices on real projects. Five minutes of preparation here pays off.

 KNOWLEDGE CHECK | Quick Check on Part 04

1. What is the difference between lossy and lossless compression?
2. When would you choose PNG over JPEG?
3. Why is JPEG a poor choice for CV training data?
4. Your team needs to store one million high resolution product photos for an e-commerce site. The site needs fast load times. Which format would you use, and why?

Part 05 | Core Image Processing Operations

Image processing operations come in three broad categories. Point operations change one pixel at a time without looking at its neighbors. Neighborhood (or filter) operations look at a small region around each pixel and combine information. Geometric operations move pixels around without changing their values. Almost every operation you will write or call belongs to one of these three families.

Point Operations

A point operation reads one pixel and writes one pixel back. The operation does not care what is happening anywhere else in the image. Examples: making an image brighter, increasing contrast, inverting colors, applying a threshold.

Brightness and Contrast

Brightness adds a constant value to every pixel. Want a brighter image? Add 30 to every pixel value. Contrast multiplies every pixel by a constant. Want more contrast? Multiply every pixel by 1.5 (and clip the result so it stays in the 0 to 255 range). These two simple operations underlie every 'auto-enhance' button on every phone camera.

Thresholding

Thresholding turns a grayscale image into a pure black and white image. Pick a threshold value, say 127. Every pixel above 127 becomes 255 (white). Every pixel below becomes 0 (black). The result is a binary image, useful for separating an object from its background. Thresholding is the first step in many classic CV tasks: document scanning, license plate reading, basic shape detection.

Gamma Correction

Gamma correction adjusts midtones without changing the darkest darks or the brightest brights. It is how cameras and displays compensate for the non-linear way humans see brightness. A small gamma adjustment can dramatically improve a dim photo's usability. CV engineers reach for gamma when an image looks 'flat' or has detail hidden in the shadows.

Neighborhood Operations: Filtering

A neighborhood operation looks at a small region (often 3 by 3 or 5 by 5 pixels) around each target pixel, combines those values somehow, and writes a new value to the output pixel. The output pixel is influenced not just by itself but by its neighbors. This is where most of the visual effects you have ever

seen in photo software come from. Blurring, sharpening, edge detection, noise removal. All neighborhood operations.

The Kernel Idea

Filtering uses a small grid of numbers called a kernel. Imagine a 3 by 3 grid of numbers. You slide this grid across the image, one pixel at a time. At each position, you multiply each kernel number by the pixel underneath it, add up the results, and put the total in the output pixel. The choice of kernel numbers determines the effect. Different kernels produce different filters. This is the core mechanic behind every image filter, and (as you will see in Module 5) it is also the core mechanic behind every convolutional neural network. The same idea scales from a \$5 photo app to the most advanced AI systems in the world.

MATH MOMENT (optional) | What a Kernel Does, Step by Step

Suppose you have a 3 by 3 averaging kernel where every value is one ninth. You slide this kernel over the image. At each spot, you multiply the nine kernel values by the nine pixel values underneath, add the nine products, and that sum becomes one pixel in the output.

If the nine pixels underneath happen to be 100, 100, 100, 100, 100, 100, 100, 100, 100 (a flat region), the output is 100. Same as input.

If the nine pixels are 0, 0, 0, 0, 200, 0, 0, 0, 0 (one bright pixel surrounded by black), the output is about 22. The bright pixel got averaged into its dark neighbors. The result is a softer, blurrier image.

Change the nine kernel numbers and you get a different filter. Make some positive and some negative and you can detect edges. Make them in a Gaussian (bell-curve) pattern and you get smooth blur. The whole field of image filtering is just choosing kernel numbers.

Blurring: Smoothing Out Noise

Blurring is the most common filter. It averages each pixel with its neighbors, smoothing out small variations. Useful for removing sensor noise, hiding minor details, and making downstream processing more stable. The most common variants are box blur (simple average), Gaussian blur (weighted average that respects how far away each neighbor is), and median blur (take the middle value of the neighbors, which is excellent at removing salt-and-pepper noise without damaging edges).

Sharpening: Boosting Detail

Sharpening does the opposite of blurring. It amplifies the difference between a pixel and its neighbors, making edges and textures pop. The trick is that sharpening can also amplify noise, so you usually want to denoise first and sharpen second.

Edge Detection

Edge detection finds places in the image where brightness changes quickly. A black-to-white boundary is a strong edge. A slow gradient is a weak edge. Solid color regions have no edges. Edge detection is the first step in countless CV tasks: shape detection, contour finding, document scanning, lane detection in self-driving systems.

The most famous edge detectors are Sobel (simple, fast, used everywhere) and Canny (more sophisticated, multi-stage, gives cleaner results). You do not need to memorize how they work internally. You need to know they exist, when to use them, and what kind of output they produce (a new image where bright pixels indicate edges and dark pixels indicate flat regions).

MISCONCEPTION ALERT | Edge Detection Is Not Magic

Students often imagine edge detection as some kind of intelligent algorithm that 'understands' object boundaries. It does not. Edge detection just finds places where pixel brightness changes quickly. A glare reflection, a shadow boundary, a printed line on the ground, and a real object edge all look the same to an edge detector. The cleverness in modern CV is in what you do with the edges, not in finding them.

Morphological Operations

Morphological operations work on binary or grayscale images and change the shape of regions. The two basic operations are erosion (shrinks bright regions) and dilation (grows bright regions). Combining them gives you opening (erosion then dilation, useful for removing small noise) and closing (dilation then erosion, useful for filling small holes in regions you care about). Morphology is heavily used in document processing, medical imaging, and any task where you have already thresholded an image and now need to clean up the shapes you found.

Geometric Operations

Geometric operations move pixels around without changing their values. The pixel that was at (100, 100) ends up at (150, 100). The colors are the same; the positions changed.

Resize

Resizing changes the dimensions of the image. Downsizing throws away information (you have fewer pixels than before). Upsizing has to invent pixels (the algorithm fills in values that were never measured). The interpolation method controls how this inventing happens; common choices are nearest neighbor

(fastest, blockiest), bilinear (smoother), and bicubic (smoothest, slowest). For CV training, you usually resize input images to a fixed size that matches what the model expects.

Crop

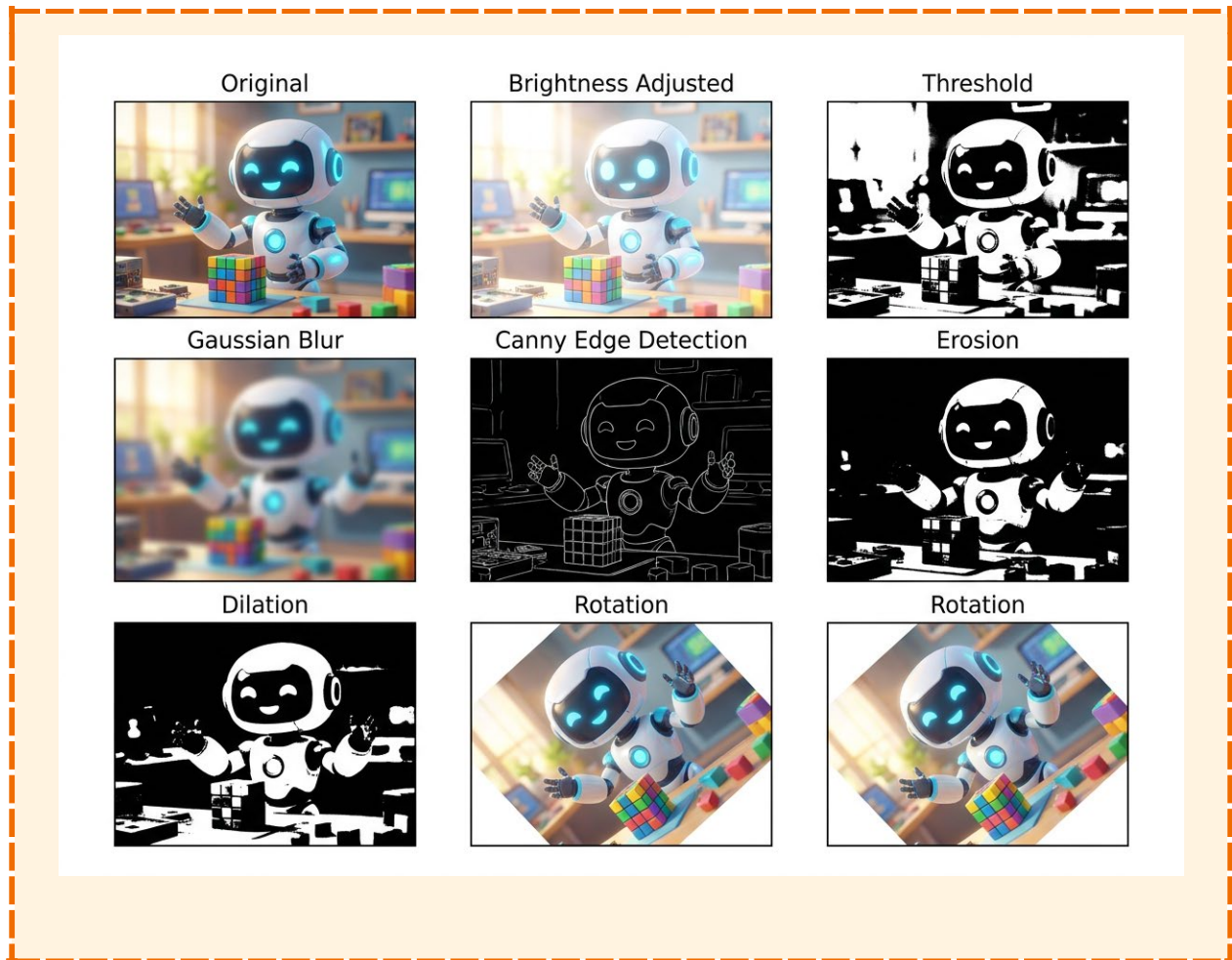
Cropping cuts out a rectangular region of the image and throws the rest away. Useful for focusing on a region of interest, for data augmentation during training, and for trimming unwanted borders.

Rotate and Flip

Rotation spins the image around a center point. Flipping mirrors the image left-right or top-bottom. Both are common data augmentation steps during model training; showing the model a flipped version of every training image effectively doubles the training data.

Perspective Transform

Perspective transform corrects the distortion when a camera is not perpendicular to a flat surface. A photo of a document taken at an angle can be straightened. A road photo from a dashcam can be projected to look like a bird's-eye view. This operation is the workhorse of document scanning apps and many self-driving perception systems.



INDUSTRY PRACTICE | A Typical Production Preprocessing Pipeline

Here is what a real preprocessing pipeline often looks like for a production object detection system.

Step 1: Read image (any format, decode to RGB).

Step 2: Resize to the model's expected input size (often 640 by 640).

Step 3: Convert BGR to RGB if read by OpenCV.

Step 4: Normalize pixel values to 0-to-1 range (divide every value by 255).

Step 5: Optionally subtract a per-channel mean (specific to the trained model).

Step 6: Convert to a tensor in the shape the model expects.

Six lines of code, half a dozen pitfalls, and an entire job category of people who specialize in getting them right.

 THINK ABOUT IT

Every filter in every photo app you have ever used is one of the operations from this part, or a chain of them. Instagram filters. Snapchat filters. The 'enhance' button on your phone. The black-and-white mode on your camera. The blur-the-background mode on your video call app. All of it is built from point operations, neighborhood operations, and geometric operations. Once you see this, you cannot unsee it.

 KNOWLEDGE CHECK | Quick Check on Part 05

1. Name the three categories of image processing operations.
2. What is a kernel? Why is it the same idea behind both photo filters and convolutional neural networks?
3. What does edge detection actually detect? What does it NOT understand?
4. A document scanner app needs to straighten a photo of a tilted receipt. Which kind of operation does it use?
5. Why is resizing not a 'free' operation when scaling up?

Part 06 | Tools of the Trade

This part covers the two libraries you will use most often in this course and on the job: OpenCV and Pillow. Both are open source, both are free, both run on Python (and other languages). They cover the same space but with different strengths. By the end of this part you will know which one to reach for when.

OpenCV: The Heavyweight

OpenCV (Open Source Computer Vision Library) is the dominant computer vision library in the industry. It was originally built by Intel in 1999, written in C and C++ for speed, with Python bindings (called cv2) added later. Today it powers everything from research labs to factory floors to self-driving cars.

OpenCV does almost everything in this course. Reading and writing images and video. Every filtering, edge detection, and morphological operation. Color space conversion. Geometric transforms. Camera calibration. Object detection. Optical flow. Tracking. Face detection. Feature extraction. Even basic deep learning inference. If a CV operation has a name, OpenCV probably has a function for it.

The trade-off is that OpenCV is large, complex, and sometimes feels old. The API is a mix of styles (some functions are very modern, some look like they were written in 2001). Error messages can be cryptic. The documentation, while extensive, can be hard to navigate. New CV engineers spend their first few weeks just learning where things live. After that, it becomes second nature and stays useful for the rest of your career.

OpenCV at a Glance

A typical first OpenCV program looks like this. The Python library is called cv2.

- cv2.imread reads an image from disk into a NumPy array.
- cv2.imshow displays an image in a window.
- cv2.imwrite saves an image to disk.
- cv2.cvtColor converts between color spaces (the BGR-to-RGB trick lives here).
- cv2.resize changes image dimensions.
- cv2.GaussianBlur, cv2.Canny, cv2.threshold are the most common processing functions.

Notice the pattern: cv2 dot function name. Hundreds of functions sit in the cv2 module. You learn them as you need them. No one memorizes them all.

 INDUSTRY PRACTICE | Why OpenCV Dominates

OpenCV won the industry standard race for three reasons. First: it is fast, because the heavy lifting happens in C++ underneath the Python wrapper. Second: it has the widest function library of any CV tool, so you rarely need to leave it. Third: it is free and open source, so every company on Earth has standardized on it. Even modern deep learning pipelines built on PyTorch or TensorFlow almost always use OpenCV for the preprocessing and post-processing around the model. OpenCV is not going anywhere.

Pillow: The Friendlier One

Pillow (often called PIL, its older name) is a pure Python image library. It is smaller in scope than OpenCV, focused on basic image manipulation rather than advanced computer vision. Reading, writing, resizing, cropping, rotating, basic filtering, format conversion. It does these things well and with a much friendlier interface than OpenCV.

Pillow is what you reach for when OpenCV would be overkill. Want to quickly resize a folder of photos? Pillow. Want to read an image and feed it to a deep learning model? Pillow's `Image.open` is more convenient than `cv2.imread` (and it uses RGB order, no BGR surprises). Want to add text to an image? Pillow's `ImageDraw` is far simpler than OpenCV's drawing functions. Lots of deep learning libraries (PyTorch, transformers, Hugging Face) default to Pillow for image input because of this simplicity.

Pillow at a Glance

A typical Pillow program is short.

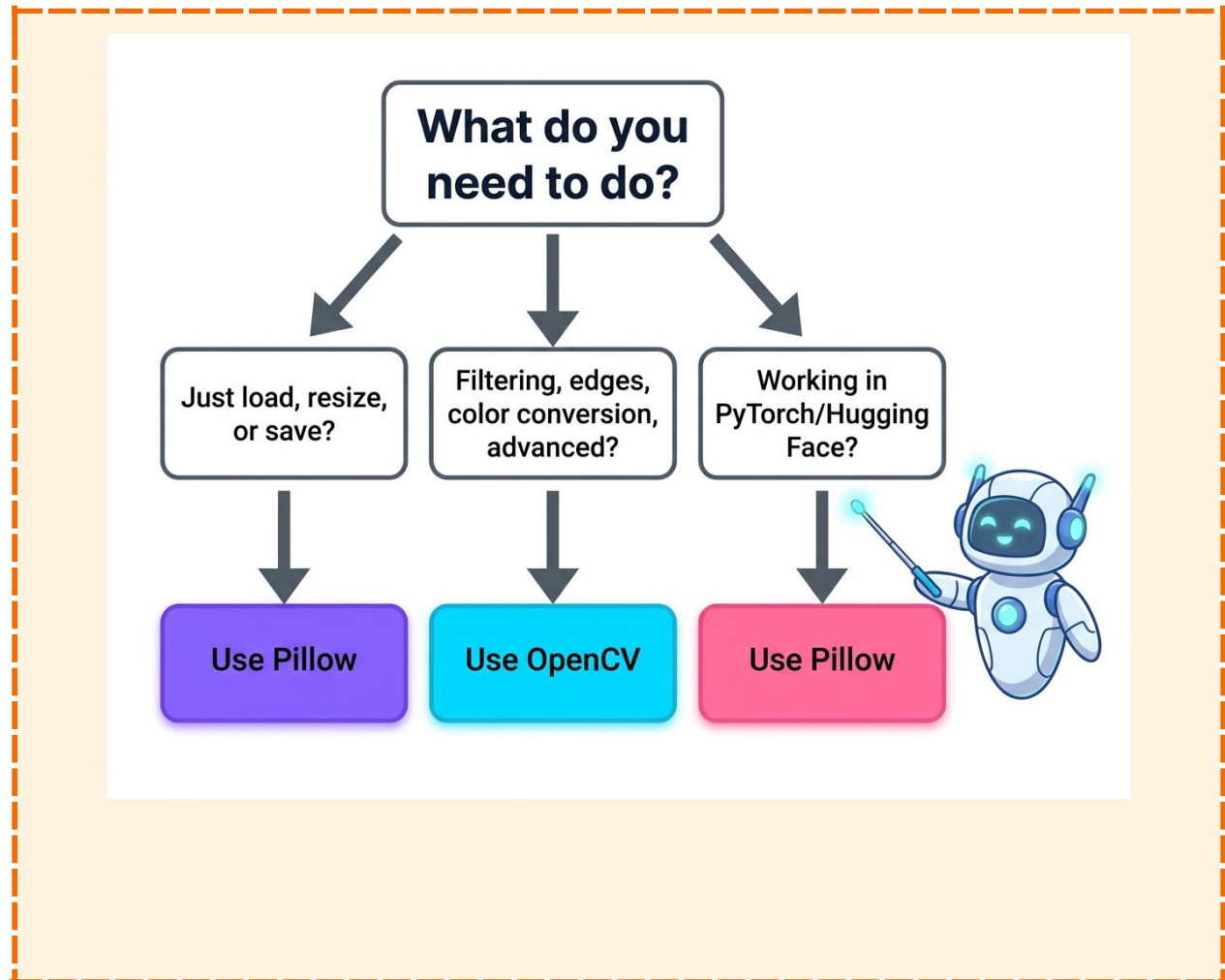
- `Image.open` loads an image as an `Image` object (not a NumPy array; you can convert it to one easily).
- `image.resize` changes dimensions.
- `image.crop` cuts out a region.
- `image.rotate` rotates the image.
- `image.save` writes the file in whatever format you specify.

OpenCV or Pillow? When To Use Which

Most CV engineers use both. The rough rule is: Pillow for simple stuff, OpenCV for everything else. If you need to do anything resembling actual computer vision (filtering, edge detection, feature extraction, anything from this module beyond basic resizing), go OpenCV. If you just need to read, resize, or save an

image quickly and you do not need any CV functions, Pillow is more pleasant. Many production pipelines use Pillow to load images and then convert them to NumPy arrays for further processing.

Other libraries exist too, and you will see them mentioned. scikit-image is a friendlier alternative to OpenCV for some scientific work. ImageMagick is a command-line tool for batch processing. NumPy underneath everything is what holds the actual pixel arrays. But OpenCV and Pillow cover most workforce needs.



 CAREER CONNECTION | OpenCV on Your Resume

OpenCV is the single most-listed skill in computer vision job postings. More than any specific deep learning framework. More than any specific model. More than any specific cloud platform. Posting 'OpenCV' on a CV resume is the table stakes; you cannot leave it off. Posting fluency in OpenCV (specific projects, specific functions used, specific bugs solved) is what gets interviews. By the end of this course you will have OpenCV experience to point to. Use it on your resume and your LinkedIn profile.

 INDUSTRY PRACTICE | The Lab You Are About To Do

Module 02's lab uses both OpenCV and Pillow to load a real image, inspect its properties (dimensions, color space, bit depth), convert it to grayscale, apply a Gaussian blur, run Canny edge detection, and save the results. Every step in that lab is something a working CV engineer does every day. Do the lab carefully. Read the code line by line. Ask why each step exists. The function calls will start feeling natural after about three or four lab sessions, and after that you will use them for the rest of your CV career.

 KNOWLEDGE CHECK | Quick Check on Part 06

1. What is OpenCV, and what is the Python module called?
2. What is the biggest gotcha about loading images in OpenCV?
3. When would you use Pillow instead of OpenCV?
4. Why is OpenCV faster than a pure Python image library?

Module 02 Review Questions

Use these questions to test your understanding before the quiz. Each one targets a key idea from this module.

Conceptual

1. Explain in your own words what 'an image is a grid of numbers' means.
2. List the three goals of image processing and give a real-world example of each.
3. Why does an 8-bit image have exactly 256 possible pixel values?
4. What is a channel in a color image? How many channels does an RGB image have?

Applied

5. Your CV pipeline accepts 224 by 224 RGB images. A user uploads a 4032 by 3024 BGR phone photo. List the preprocessing steps you must apply before the model can see it.
6. You are building a traffic light detector. The system must recognize red, yellow, and green lights regardless of weather and time of day. Which color model would you use and why?
7. Pick the right format for each scenario: (a) photo backups on a phone, (b) icons on a website, (c) medical scans for archive, (d) training data for a deep learning model.
8. Explain why both image filtering and convolutional neural networks use the same core idea of a sliding kernel.

Reflection

9. Name one image processing concept that surprised you. Why?
10. In your own words, what mistake have you seen (in software, on a phone, in a photo app) that might have come from a preprocessing error? Describe what could have gone wrong.

Glossary

Quick reference for the terms introduced in this module.

Pixel | The smallest unit of a digital image. Holds a brightness number (grayscale) or a set of numbers (color).

Resolution | The width and height of the pixel grid (for example 1920 by 1080).

Bit depth | How many possible values each pixel can take. 8 bit allows 256 values; 16 bit allows 65,536.

Channel | One of the values per pixel in a multi-value image. A color RGB image has three channels.

Grayscale | An image with one channel per pixel, representing brightness only.

RGB | A color model with three channels: red, green, and blue. The default for most digital images.

BGR | OpenCV's default channel order. Same data as RGB but stored in reverse order.

HSV | A color model with three channels: hue, saturation, and value. Useful for color-based filtering.

LAB | A perceptually uniform color model. Equal numeric differences correspond to equal visual differences.

JPEG | The most common image format. Lossy. Best for photographs.

PNG | Lossless image format with transparency support. Best for logos, icons, screenshots, and CV training data.

TIFF | Flexible professional format. Supports lossless, multi-page, and high bit depth.

Lossy compression | Compression that throws away some information to make files smaller. JPEG is lossy.

Lossless compression | Compression that preserves every pixel exactly. PNG is lossless.

Point operation | An image processing operation that modifies one pixel at a time without looking at neighbors.

Neighborhood operation | An image processing operation that uses surrounding pixels to compute each output pixel. Also called filtering.

Geometric operation | An image processing operation that moves pixels around without changing their values. Resize, rotate, crop.

Kernel | A small grid of numbers that slides across the image during filtering. The numbers determine the filter's effect.

Threshold | An operation that turns a grayscale image into binary (black and white) based on a cutoff value.

Edge detection | Finding places in the image where brightness changes quickly. Sobel and Canny are the most common methods.

Morphological operation | A shape-changing operation. Erosion shrinks regions, dilation grows them.

Gaussian blur | A smoothing filter that gives nearby pixels more influence than distant ones.

OpenCV | The dominant open source computer vision library. Python module is cv2.

Pillow (PIL) | A friendlier Python image manipulation library. Best for simple read, resize, save tasks.

Interpolation | The method used to invent new pixel values when resizing. Nearest, bilinear, and bicubic are the common choices.

References and Further Reading

Documentation and Tutorials

OpenCV Documentation. <https://docs.opencv.org>. The primary reference for every OpenCV function. Bookmark this.

Pillow Documentation. <https://pillow.readthedocs.io>. The official Pillow guide.

scikit-image User Guide. https://scikit-image.org/docs/stable/user_guide.html. A friendly Python alternative for some operations.

Foundational Books

Szeliski, R. (2022). Computer Vision: Algorithms and Applications, 2nd edition. Springer. A free PDF is available from the author's website. Comprehensive and authoritative; treats classical image processing alongside modern methods.

Gonzalez, R. C. and Woods, R. E. (2018). Digital Image Processing, 4th edition. Pearson. The classic textbook on the mathematics and algorithms of image processing.

Practical Learning Resources

PyImageSearch. <https://pyimagesearch.com>. Adrian Rosebrock's tutorials are widely used by working CV engineers. Strong on OpenCV applications.

LearnOpenCV. <https://learnopencv.com>. Practical OpenCV tutorials by Satya Mallick. Frequently updated for modern workflows.

OpenCV Tutorials on YouTube. The official OpenCV channel hosts short videos covering most common operations.

Industry and Career Context

Anthropic (2026). Labor market impacts of AI: A new measure and early evidence. Anthropic Economic Index. Useful context for CV career trajectories in the broader AI labor market.

Stack Overflow Developer Survey. Annual snapshot of which tools and libraries professional developers actually use. OpenCV consistently appears in CV-related cuts of the data.

Looking Ahead: Module 03 Preview

Module 03 introduces classic machine learning for computer vision. Before deep learning conquered the field, CV systems were built using handcrafted features and traditional classifiers. Understanding this approach matters for three reasons.

First, it teaches you what features actually are. Modern neural networks learn features automatically, but the concept of a 'feature' (a measurable property of an image that captures something useful) was invented in this older era. Without the vocabulary, the modern stuff makes less sense.

Second, classic ML is still in production. Medical imaging systems certified by the FDA. Embedded vision on resource-constrained devices. Quality inspection in factories where decisions must be explainable. Many of these still rely on classic methods because they are interpretable, fast, and work with small datasets.

Third, it sets the stage for understanding why deep learning was such a big deal when it arrived. Without the contrast, you cannot appreciate the leap.

Specifically in Module 03 you will meet feature extraction techniques like HOG (histogram of oriented gradients), SIFT (scale-invariant feature transform), SURF, and LBP (local binary patterns). You will see how classifiers like Support Vector Machines, Random Forests, and k-Nearest Neighbors were combined with these features to build object recognizers. And you will trace the path that eventually led to convolutional neural networks taking over.

Questions about this module? email or message me if you have any questions.